

Snake Pi: Jogo Interativo em Raspberry Pi com Integração de Periféricos

Ana Paula Arejano¹, Carolina Tavares Duarte¹, Paulo Henrique Mota de Oliveira¹

¹ Escola Politécnica da Universidade de São Paulo (EP-USP)
Laboratório de Processadores
São Paulo - SP - Brasil

carolinatavaresduarte@usp.br, aparejano@usp.br, paulo.mota028@usp.br

Abstract. *This paper presents Snake Pi, an embedded version of the classic Snake game developed for a Raspberry Pi 4 coupled with a Freenove Projects Board for Raspberry Pi v1.2. The game is rendered on an HDMI display and integrates physical inputs and outputs, including push buttons, an analog-to-digital converter, a potentiometer, buzzers, an LCD, a four-digit seven-segment display driven by a 74HC595 shift register, and an optional RFID reader. We designed the software as independent layers for game logic, rendering, hardware access, shared state, and peripheral threads. This separation enables simulation without physical hardware and automated validation of the core rules. The final test suite contains 34 automated tests, all of which passed. We also discuss hardware constraints, such as the multiplexing requirements of the four-digit display and GPIO conflicts, and the strategies adopted to preserve responsiveness and graceful degradation.*

Resumo. *Apresentamos o Snake Pi, uma versão embarcada do jogo clássico Snake, desenvolvida em uma Raspberry Pi 4 acoplada à Freenove Projects Board for Raspberry Pi v1.2. O jogo é renderizado em um monitor HDMI e integra entradas e saídas físicas, incluindo botões, conversor analógico-digital, potenciômetro, buzzers, LCD, display de quatro dígitos e sete segmentos controlado por registrador 74HC595 e leitor RFID opcional. Organizamos o software em camadas independentes para lógica do jogo, renderização, acesso ao hardware, estado compartilhado e threads periféricas. Essa separação permite executar o sistema em modo de simulação e testar automaticamente as regras centrais sem a placa física. A suíte final contém 34 testes automatizados, todos aprovados. Também discutimos restrições encontradas na placa, como a multiplexação do display de quatro dígitos e conflitos entre GPIOs, bem como as estratégias adotadas para manter responsividade e degradação graciosa.*

1. Introdução e motivação

O projeto Snake Pi foi desenvolvido na disciplina Laboratório de Processadores com o propósito de integrar conceitos de programação, arquitetura de software, concorrência e interação com periféricos em uma plataforma embarcada. Escolhemos o jogo Snake porque sua lógica é suficientemente clara para permitir a validação sistemática do comportamento, mas também oferece espaço para incorporar entradas físicas, feedback visual e sonoro, persistência e diferentes modos de dificuldade.

Após avaliarmos os recursos efetivamente disponíveis na Freenove Projects Board e os prazos do projeto, definimos o escopo: utilizamos botões físicos como direcional, potenciômetro para ajuste de velocidade, buzzers para retorno sonoro, monitor HDMI para a interface principal e periféricos adicionais para espelhar o estado da partida.

A documentação e os exemplos oficiais da Freenove foram utilizados como referência para a identificação dos periféricos e da pinagem da placa [Freenove 2026b, Freenove 2026a]. Para a interface gráfica, adotamos o Pygame, biblioteca voltada a aplicações multimídia e jogos em Python [Pygame Community 2026]. O acesso a dispositivos GPIO foi abstraído principalmente com GPIO Zero [Nuttall et al. 2026].

1.1. Projetos e materiais de inspiração

Durante a definição do escopo, consultamos projetos de Snake desenvolvidos para Raspberry Pi e Sense HAT, especialmente uma atividade oficial da Raspberry Pi Foundation e uma implementação pública em Python para a matriz de LEDs e o joystick do Sense HAT [Raspberry Pi Foundation 2026, Duller 2016]. Esses materiais serviram como referência conceitual para a interação física e para as regras básicas do jogo. A arquitetura em camadas, a integração com a Freenove Projects Board, o uso de threads e a estratégia de testes foram desenvolvidos especificamente para o Snake Pi.

2. Objetivos

2.1. Objetivo geral

Nosso objetivo geral foi desenvolver uma versão interativa do jogo Snake executada integralmente em uma Raspberry Pi, com interface gráfica em monitor HDMI e integração com periféricos físicos da Freenove Projects Board.

2.2. Objetivos específicos

Definimos os seguintes objetivos específicos:

1. implementar movimento, crescimento, frutas, colisões, níveis, dificuldades e ranking persistente;
2. separar a lógica do jogo das dependências de Pygame e GPIO;
3. utilizar os botões coloridos da placa como controle direcional;
4. utilizar o botão do joystick para confirmação e pausa;
5. utilizar o potenciômetro para alterar a velocidade da cobra;
6. produzir retorno sonoro com buzzers ativo e passivo;
7. controlar o LCD e o display de quatro dígitos associado ao 74HC595;
8. permitir seleção opcional de dificuldade por RFID;
9. executar periféricos lentos em threads independentes;
10. permitir execução simulada e testes sem hardware;
11. manter o sistema operante mesmo quando um periférico estiver ausente.

3. Especificação de requisitos

Os requisitos foram refinados ao longo das entregas semanais. Em particular, distinguimos funcionalidades implementadas no software de funcionalidades validadas fisicamente na configuração final da placa.

3.1. Requisitos funcionais

ID	Requisito	Status
RF01	Exibir menu com início da partida, ranking, configurações e saída.	Atendido
RF02	Permitir movimento da cobra em quatro direções.	Atendido
RF03	Fazer a cobra crescer ao consumir uma fruta.	Atendido
RF04	Gerar frutas somente em células livres.	Atendido
RF05	Detectar colisão com parede conforme a dificuldade.	Atendido
RF06	Detectar colisão da cobra com o próprio corpo.	Atendido
RF07	Exibir Game Over e permitir reinício ou retorno ao menu.	Atendido
RF08	Controlar a cobra por botões físicos, com teclado como alternativa.	Atendido
RF09	Usar o botão do joystick para confirmar opções e pausar a partida.	Atendido
RF10	Exibir pontuação e nível no LCD1602 quando o dispositivo estiver disponível.	Implementado
RF11	Permitir seleção opcional de dificuldade por RFID.	Implementado
RF12	Emitir efeitos sonoros distintos para eventos do jogo.	Atendido
RF13	Exibir a pontuação no display de quatro dígitos e sete segmentos, controlado pelo 74HC595.	Implementado
RF14	Ajustar a velocidade da cobra pelo potenciômetro.	Atendido
RF15	Aumentar progressivamente a dificuldade por níveis.	Atendido
RF16	Gerar frutas especiais temporárias.	Atendido
RF17	Gerar obstáculos de acordo com a dificuldade.	Atendido
RF18	Persistir e ordenar o ranking em arquivo JSON.	Atendido

3.2. Requisitos não funcionais

Os requisitos não funcionais foram especificados com base em características de qualidade da ISO/IEC 25010, particularmente eficiência de desempenho, confiabilidade, usabilidade, manutenibilidade e portabilidade [International Organization for Standardization 2011].

ID	Requisito	Característica
RNF01	Executar em Raspberry Pi OS Bookworm e Python 3.11.	Portabilidade
RNF02	Renderizar a interface em monitor HDMI sem bloquear o processamento de entradas.	Eficiência
RNF03	Manter o jogo operante quando um periférico opcional falhar ou estiver ausente.	Confiabilidade
RNF04	Organizar o código em camadas com baixo acoplamento.	Manutenibilidade
RNF05	Permitir testes da lógica sem acesso ao hardware físico.	Testabilidade
RNF06	Centralizar pinagem e parâmetros em um único módulo de configuração.	Modificabilidade
RNF07	Utilizar type hints, docstrings e convenções compatíveis com PEP 8.	Analísabilidade
RNF08	Executar operações periféricas lentas sem bloquear o laço principal.	Eficiência
RNF09	Encerrar threads e liberar GPIOs de forma controlada.	Confiabilidade
RNF10	Atualizar de forma estável o display de quatro dígitos controlado pelo 74HC595, respeitando as limitações de multiplexação da placa.	Compatibilidade

4. Arquitetura do sistema

4.1. Arquitetura física

A Raspberry Pi concentra todo o processamento. O monitor HDMI apresenta a interface gráfica principal, enquanto a placa Freenove reúne as entradas e saídas físicas. O ADS7830 fornece conversão analógico-digital para o potenciômetro e outras entradas analógicas. O LCD utiliza I²C, o RFID opcional utiliza SPI e os botões e buzzers utilizam GPIOs diretos.

A matriz 8×8, o bar graph e o display de quatro dígitos compartilham sinais associados ao 74HC595. Na configuração observada, o uso simultâneo desses dispositivos depende da posição das chaves DIP e da topologia física da placa. Por isso, implementamos um gerenciador que trabalha em modo exclusivo com apenas um componente por vez. Demos prioridade ao display de quatro dígitos, uma vez que era o mais importante para a jogabilidade por mostrar a pontuação no jogo.

4.2. Arquitetura de software

Organizamos o código em módulos de lógica, hardware, renderização, threads, utilitários e configuração. A lógica não depende de Pygame ou GPIO, e o acesso aos periféricos não contém regras do jogo. O arquivo `main.py` realiza a injeção das dependências e coordena o ciclo principal.

O estado compartilhado é protegido por `RLock`. As threads periféricas leem cópias imutáveis por meio de *snapshots*, evitando manter a trava durante operações de entrada e saída. Eventos sonoros e animações são encaminhados por filas não bloqueantes.

4.3. Modelagem comportamental

A aplicação possui os estados `MENU`, `PLAYING`, `PAUSED`, `GAME_OVER`, `HIGH_SCORES`, `SETTINGS` e `QUIT`. A Figura 3 apresenta as principais transições.

4.4. Diagrama de sequência

A Figura 4 apresenta o ciclo temporal de uma partida. O laço principal coleta as entradas, atualiza o motor e publica o estado. O renderizador consulta um snapshot para desenhar a tela, enquanto as threads periféricas atualizam as saídas físicas. Em caso de colisão, o motor encerra a partida e o placar persiste o resultado.

5. Ferramentas, linguagens e hardware

5.1. Linguagens e ferramentas

Tabela 3. Linguagens e ferramentas utilizadas.

Item	Uso
Python 3.11	Implementação do jogo, drivers, concorrência e testes.
Pygame	Renderização no monitor HDMI e tratamento de eventos.
D2	Produção dos diagramas editáveis.
Markdown e L ^A T _E X	Documentação incremental e relatório final.
Git e GitHub	Versionamento, releases, pull requests e avaliação por pares.
TeXstudio	Edição e compilação do relatório no modelo da SBC.

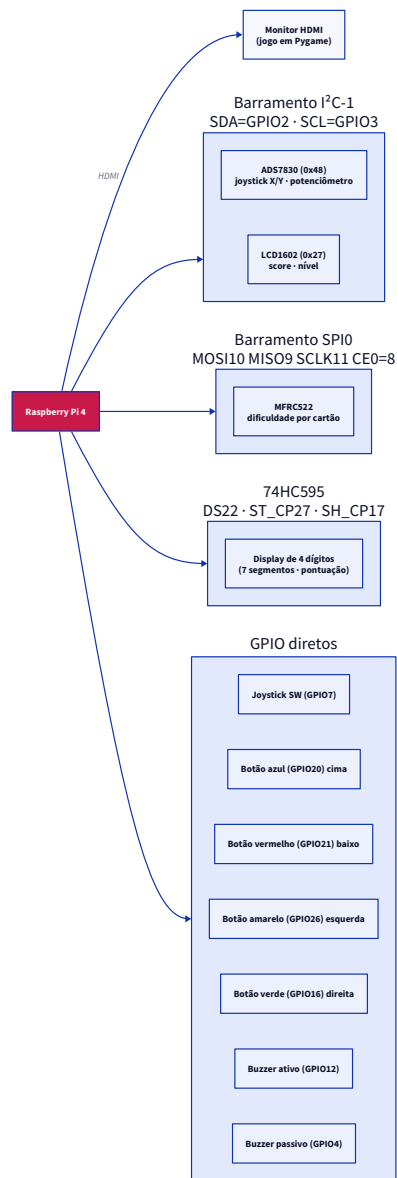


Figura 1. Arquitetura física do Snake Pi.

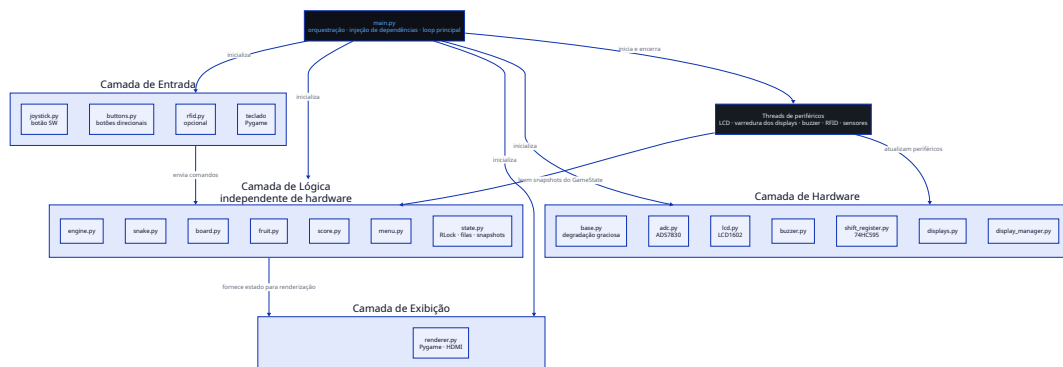


Figura 2. Arquitetura de software e separação entre os módulos.

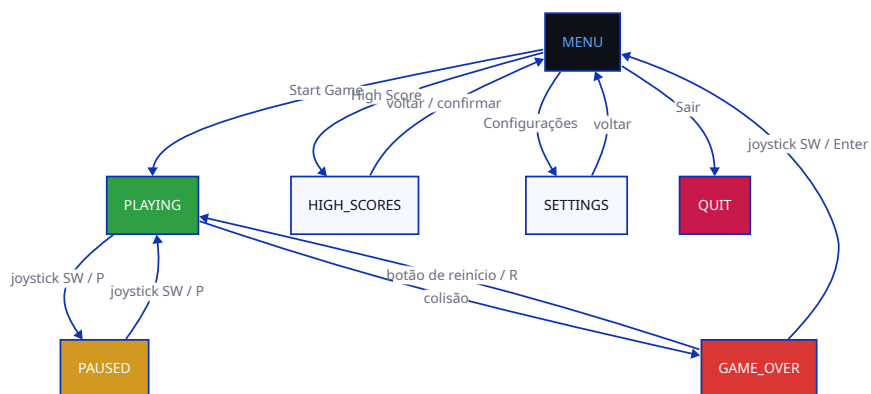


Figura 3. Máquina de estados da aplicação.

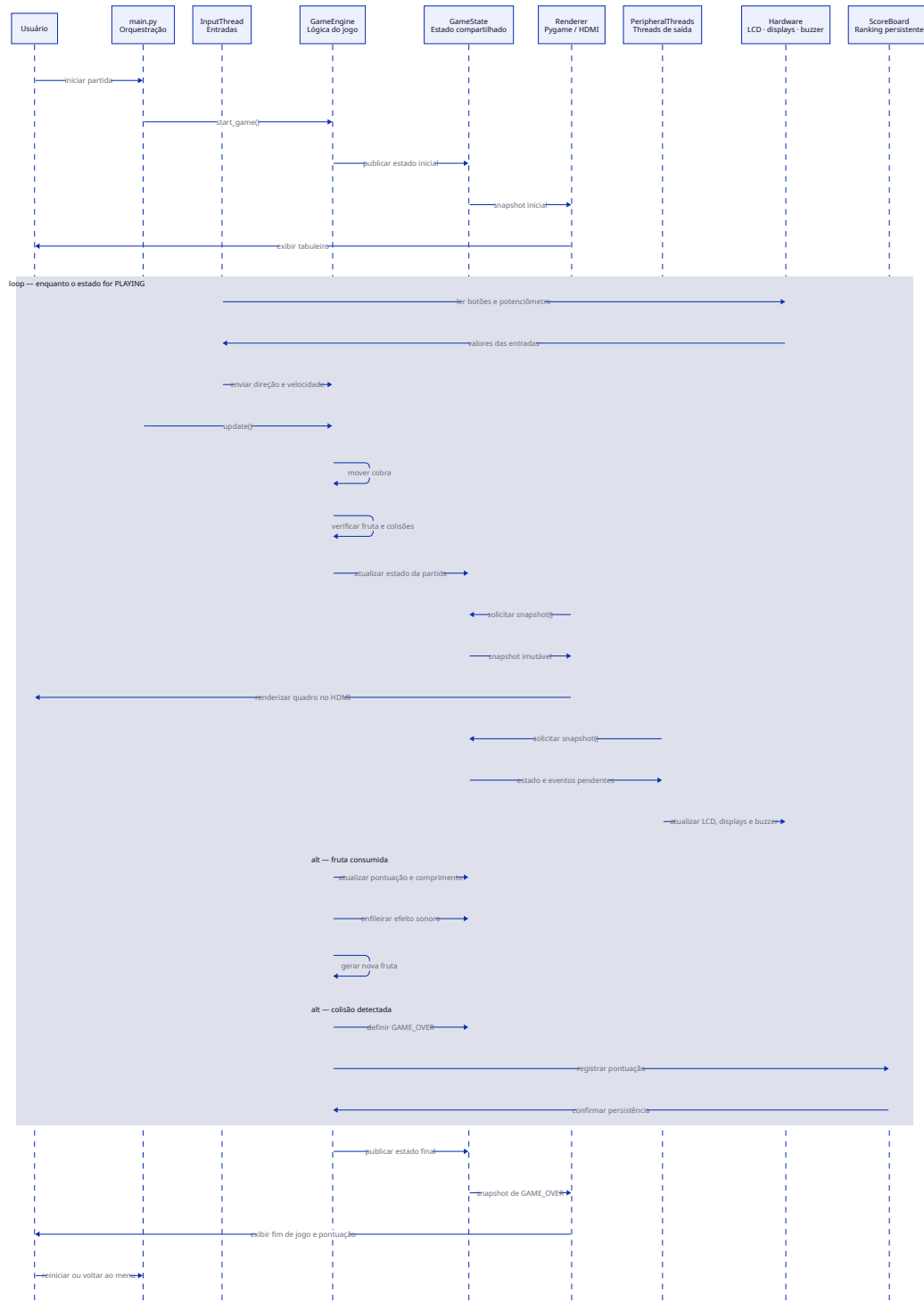


Figura 4. Diagrama de sequência do ciclo principal da partida.

5.2. Bibliotecas e frameworks

Tabela 4. Principais bibliotecas do projeto.

Biblioteca	Finalidade
pygame	Interface gráfica, teclado, janela e controle de quadros.
gpiozero	Botões, buzzers e dispositivos digitais.
lgpio	Backend de GPIO no Raspberry Pi OS Bookworm.
smbus/smbus2	Comunicação I ² C com ADS7830 e LCD.
spidev e mfrc522	Comunicação opcional com o leitor RFID.
threading e queue	Concorrência, eventos e filas não bloqueantes.
unittest	Testes automatizados.

5.3. Hardware

Utilizamos Raspberry Pi 4, Freenove Projects Board v1.2, monitor HDMI, botões coloridos, potenciômetro, buzzers ativo e passivo, ADC ADS7830, LCD1602, display de quatro dígitos e leitor RFID opcional. A documentação da Freenove descreve os recursos do kit e fornece exemplos de uso em Python [[Freenove 2026b](#)].

6. Metodologia de desenvolvimento

Adotamos desenvolvimento incremental ao longo de quatro semanas, acompanhando o cronograma da disciplina.

Tabela 5. Evolução semanal do projeto.

Semana	Atividade principal	Evidência
1	Estruturação do repositório e implementação inicial da lógica do jogo.	Release v1.0 .
2	Implementação dos módulos dos periféricos, scripts de diagnóstico e avaliação por pares por meio de GitHub Issues.	Release v2.0 e Issues da revisão por pares .
3	Integração da lógica, da interface gráfica e dos periféricos por meio de threads.	Release v3.0 .
4	Consolidação do código, relatório, apresentação e demonstração final.	Release v4.0 .

A lógica foi implementada antes da integração física, o que permitiu testar movimento, colisões, frutas, pontuação, menus e persistência em qualquer computador. Em seguida, integramos os drivers individualmente e criamos modo de simulação para todos os componentes.

Também desenvolvemos scripts de diagnóstico para identificar endereços I²C, validar botões e buzzers e investigar a topologia do 74HC595. As decisões encontradas na bancada foram centralizadas em `config.py`. Durante o desenvolvimento, utilizamos

commits incrementais, branches e pull requests. Na entrega final, o repositório contém apenas a branch `main`, conforme a orientação da disciplina.

O repositório público do projeto está disponível em [repositório público no GitHub](#).

6.1. Avaliação por pares

Na segunda semana do projeto participamos da atividade de avaliação por pares proposta pela disciplina. Revisamos a especificação do projeto *Snake-Game-esp32*, desenvolvido para ESP32-C3, registrando sugestões por meio de GitHub Issues ([Issues da revisão por pares](#)).

A revisão concentrou-se principalmente em aspectos técnicos relacionados ao hardware, consistência entre requisitos e critérios de teste. Entre as contribuições realizadas destacam-se a identificação de possíveis conflitos com pinos de *strapping*, inconsistências entre requisitos de latência e o período do jogo, divergências na documentação das velocidades, definição do enlace entre Raspberry Pi e ESP32-C3 e sugestões para ampliar a cobertura dos testes automatizados.

Nosso projeto também recebeu sugestões registradas em [Issues recebidas pelo Grupo P](#), envolvendo melhorias de usabilidade, apresentação da interface e evolução do modo difícil. As sugestões foram analisadas e consideradas no planejamento das próximas evoluções do projeto.

7. Estratégia de validação

Combinamos testes automatizados, simulação ponta a ponta e experimentos em hardware. A suíte automatizada validou a lógica pura, as conversões dos displays, o ranking, os menus e o estado compartilhado.

7.1. Casos de teste e rastreabilidade

Requisito	Casos de teste ou evidência	Resultado
RF01	<code>TestMenu.test_navegacao_circular</code> e <code>test_repeticao_temporizada</code> .	Aprovado
RF02–RF03	<code>TestSnake.test_movimento</code> e <code>test_crescimento_preserva_cauda</code> .	Aprovado
RF04, RF16	Testes de geração, expiração e ausência de espaço em <code>TestFruit</code> .	Aprovado
RF05	<code>TestEngine.test_morre_na_parede</code> e <code>test_easy_atravessa_parede</code> .	Aprovado
RF06	<code>TestSnake.test_colisao_consigo_mesma</code> .	Aprovado
RF13	Testes de polaridade, multiplexação e representação em <code>TestDisplays</code> .	Aprovado
RF14–RF15	Testes do potenciômetro e das velocidades por dificuldade.	Aprovado
RF17	Testes de quantidade e área livre de obstáculos.	Aprovado
RF18	Testes de persistência, ordenação, limite e arquivo corrompido.	Aprovado
RNF03	Inicialização sem LCD/RFID, tratamento de JSON corrompido e degradação graciosa.	Aprovado
RNF05	Execução dos 34 testes sem acesso ao hardware físico.	Aprovado
RNF09	Teste de sinalização de encerramento do estado compartilhado.	Aprovado

7.2. Resultados dos testes automatizados

Executamos 34 testes automatizados. Todos foram aprovados em aproximadamente 0,04 segundo na execução registrada. A suíte pode ser reproduzida com:

```
python3 tests/test_game_logic.py -v
```

7.3. Testes em hardware

A execução registrada em log confirmou inicialização do ADC ADS7830, botão do joystick, buzzers, botões coloridos e barramento 74HC595. Na mesma execução, o LCD não foi detectado nos endereços configurados e o RFID permaneceu desabilitado, o que demonstrou a estratégia de degradação graciosa: a partida continuou normalmente sem esses dispositivos.

Tabela 7. Resultados observados na execução registrada.

Componente	Resultado observado	Situação
ADS7830	Detectado no endereço I ² C 0x48.	Validado
Botões coloridos	GPIOs 20, 21, 26 e 16 inicializados.	Validado
Botão do joystick	Inicializado no GPIO 7.	Validado
Buzzers	Buzzer ativo e passivo inicializados.	Validado
74HC595 (display de 4 dígitos)	Inicializado nos GPIOs 22, 27 e 17.	Validado
Potenciômetro	Leitura pelo ADC aplicada à velocidade.	Validado no jogo
RFID	Mantido desabilitado por padrão para liberar o GPIO 7, usado pelo botão do joystick.	Opcional / não utilizado

Os vídeos de funcionamento e as evidências visuais da bancada estão disponíveis em [pasta de vídeos no Google Drive](#). Antes da entrega, devemos conferir se a pasta está compartilhada com permissão de leitura para qualquer pessoa com o link.

8. Resultados e discussão

A arquitetura modular permitiu executar o jogo mesmo quando alguns periféricos não estavam disponíveis. Essa propriedade foi observada na bancada: a ausência do LCD e a desativação do RFID não interromperam o menu, a simulação, a renderização ou os demais periféricos.

O desacoplamento entre taxa de quadros e velocidade da cobra foi implementado por meio de acumulador temporal. Assim, atrasos de renderização não alteram diretamente a dinâmica da partida. Operações potencialmente bloqueantes, como escrita no LCD e reprodução de melodias, foram transferidas para threads específicas.

A maior dificuldade física foi o compartilhamento do barramento 74HC595. Os displays multiplexados exigem varredura frequente e podem apresentar cintilação dependendo do escalonamento do sistema operacional. Para contornar esse problema, implementamos

varredura contínua e a possibilidade de selecionar um único dispositivo físico por vez. O bar graph é a alternativa mais estável porque não depende de multiplexação, mas utilizamos o display de 4 dígitos pela sua maior utilidade ao jogador.

Outro conflito relevante envolveu o GPIO 7, usado pelo botão do joystick e associado ao CE1 do SPI. Mantivemos o RFID desabilitado por padrão e documentamos a configuração necessária para sua ativação. Essas limitações foram registradas no guia de solução de problemas e incorporadas às decisões de arquitetura.

9. Conclusões

Concluimos que o projeto cumpriu o objetivo de integrar uma aplicação interativa a diferentes periféricos de uma plataforma embarcada. Implementamos um jogo completo, com menu, dificuldades, níveis, frutas especiais, obstáculos, ranking persistente, entrada física, retorno sonoro e interface HDMI.

A separação entre lógica, interface gráfica, hardware e threads aumentou a testabilidade e a manutenibilidade. Os 34 testes automatizados foram aprovados, e os experimentos em bancada validaram os principais dispositivos utilizados. A degradação graciosa também se mostrou efetiva, pois o sistema continuou funcionando quando LCD e RFID não estavam disponíveis na execução registrada.

As principais dificuldades foram compreender a topologia real da placa, conciliar GPIOs compartilhados e manter estabilidade nos displays multiplexados. Como lições aprendidas, destacamos a importância de validar o hardware antes de fixar a arquitetura, centralizar configurações, registrar evidências e preservar a independência da lógica de domínio.

Como trabalhos futuros, propomos validar o RFID em conjunto com o botão do joystick, revisar a conexão do LCD, adicionar novos modos de jogo, utilizar termistor e fotorresistor e permitir a inserção de iniciais no ranking pelo teclado matricial.

References

- Duller, D. (2016). Snake game for the raspberry pi sense hat. https://github.com/dadul96/SnakeGame_RaspberryPi_SenseHat. Repositório de código. Acesso em: 5 ago. 2026.
- Freenove (2026a). Freenove projects kit for raspberry pi. https://github.com/Freenove/Freenove_Projects_Kit_for_Raspberry_Pi. Repositório com exemplos e recursos do kit. Acesso em: 5 ago. 2026.
- Freenove (2026b). Freenove projects kit for raspberry pi: Fnk0054 documentation. <https://docs.freenove.com/projects/fnk0054/en/latest/>. Acesso em: 5 ago. 2026.
- International Organization for Standardization (2011). Iso/iec 25010:2011: Systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models.
- Nuttall, B., Jones, D., et al. (2026). Gpio zero documentation. <https://gpiozero.readthedocs.io/>. Acesso em: 5 ago. 2026.
- Pygame Community (2026). Pygame documentation. <https://www.pygame.org/docs/>. Acesso em: 5 ago. 2026.

Raspberry Pi Foundation (2026). Slug!: Create a version of the classic snake game on a sense hat. <https://projects.raspberrypi.org/en/projects/slug>. Acesso em: 5 ago. 2026.